GENERALIDADES de los LENGUAJES FORMALES

CONCEPTOS PREVIOS

Informática Teórica: confluencia de investigaciones sobre los fundamentos de las matemáticas, teoría de máquinas, lingüística, etc. Ciencia multidisciplinar, apoyada en la observación de que los mismos fenómenos pueden aparecer y explicar áreas completamente distintas y en apariencia sin conexión.
La aparición de estudios sobre la Informática Teórica se remonta a principios de siglo del siglo XX. Dentro de los hitos más importantes se pueden citar los siguientes:

- ✓ En 1900 **David Hilbert** propone "*el problema de la decisión*", que trata de descubrir un método general para decidir si una fórmula lógica es verdadera o falsa.
- ✓ Aparición del artículo de **Kurt Gödel**, publicado en 1931, "*On Formally Undecidible Propositions in Principia Mathematica and Relates Systems*". Este trabajo puede considerarse como la mayor realización matemática del siglo XX. El teorema de Gödel sostiene que cualquier teoría matemática contendrá afirmaciones "indecidibles".
- ✓ En 1937, el matemático inglés **Alan Mathison Turing** publica un famoso artículo que desarrolla el teorema de Gödel, y que puede considerarse como el origen oficial de la Informática Teórica. En este artículo se introduce el concepto de *máquina de Turing*, una entidad matemática abstracta que formaliza por primera vez el concepto de algoritmo.
  Al ser la máquina de Turing un <u>modelo ideal de máquina capaz de adoptar una infinidad de estados posibles</u>, resulta obvio pensar que su realización está fuera del alcance práctico. Sin embargo, debido a la gran capacidad de las computadoras, la máquina de Turing resulta ser un modelo adecuado de su funcionamiento.
- ✓ En 1938, se produce un importante aporte a la Informática Teórica, en el terreno de la Ingeniería Eléctrica. Publica su artículo "A Symbolic Analysis of Relay and Switching Circuits" el matemático Claude Elwood Shannon. En él se establecen las bases para la aplicación de la lógica matemática a los circuitos electrónicos. En las décadas siguientes las ideas de Shannon se desarrollaron ampliamente, dando lugar a la formalización de una teoría sobre máquinas secuenciales y autómatas finitos.
  Desde su nacimiento, la teoría de autómatas ha encontrado aplicación en muy diversos campos. Esto se debe a que resulta muy natural considerar, tanto los autómatas como las máquinas secuenciales, sistemas capaces de transmitir (procesar) información. En definitiva, esto es equiparable a cualquier sistema existente en la naturaleza, que recibe señales de su entorno, reacciona ante ellas y emite así nuevas señales al ambiente que le rodea.
- ✓ En 1950 se vincula la Informática Teórica con otro terreno con el que habitualmente ni se había relacionado: la Lingüística. En este año el lingüista Noam Chomsky publica su "Teoría de las Gramáticas Transformacionales", que establece las bases de la lingüística matemática. Proporcionó una herramienta de inestimable ayuda, no sólo en el estudio de los lenguajes naturales, sino también en la formalización de los lenguajes de ordenador, que comenzaban a aparecer en estos años.

  Chomsky argumentaba que la **sintaxis** (la forma) es independiente de la **semántica** (el significado). Su ejemplo más famoso fue: "*Colorless green ideas sleep furiously*" (Ideas verdes incoloras duermen furiosamente). Frase sin sentido pero gramaticalmente correcta. Para probar esto, Chomsky definió las gramáticas como un sistema de reglas de reescritura:
  - **Símbolos:** Un alfabeto de palabras y categorías (Sustantivo, Verbo, etc.).
  - **Reglas de producción:** Instrucciones que dicen cómo transformar un símbolo en otro.

Aunque Chomsky buscaba entender el cerebro humano hubo quienes (como John Backus) se dieron cuenta de que sus formalismos eran perfectos para definir lenguajes artificialesLa teoría de Lenguajes Formales resultó tener una sorprendente relación con la Teoría de las Máquinas Abstractas. Los mismos fenómenos aparecían en ambas disciplinas y era posible establecer correspondencias muy claras entre ellas.

Chomsky clasificará los lenguajes formales de acuerdo a una jerarquía de cuatro niveles.

- ✓ Los lenguajes de tipo 3, o lenguajes regulares, son los que presentan una estructura más sencilla.
- ✓ Luego, aparecen los lenguajes de tipo 2, o lenguajes "independientes del contexto", suficientes para representar la sintaxis de los lenguajes de programación
- ✓ Los lenguajes de tipo 1, también llamados lenguajes "sensibles al contexto", no son de interés en la construcción de traductores
- ✓ El tipo de lenguaje más general será de tipo 0, y no posee restricción alguna.

Resulta relevante observar cómo paralelamente a la jerarquía de lenguajes aparece otra de máquinas abstractas equivalentes, como se observa en el esquema siguiente:

	LENGUAJES	MAQUINA
-	Lenguajes Tipo 3	Autómatas Finitos
	Lenguajes Tipo 2	Autómatas de Pila
+	Lenguajes Tipo 1	Autómatas Linealmente Acotados
	Lenguajes Tipo 0	Máquinas de Turing

La fecha roja indica la complejidad creciente que tienen los lenguajes en cuanto a la potencia generadora, como más adelante analizaremos.

FORMALISMOS, DEFINICIONES y METALENGUAJE

☞ **LENGUAJES**

Se llama **lenguaje** a un conjunto de cadenas

☞ **ALFABETOS**

Se llama **alfabeto** a un conjunto finito, no vacío, cuyos elementos se denominan *símbolos*. Los alfabetos se definen mediante la enumeración de los símbolos que contiene. Ejemplos:

A1={A, B, C, D, E, F , G, ..., Z}
A2={0,1}
A3={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, .}
A4={\ , /, $, @}

☞ **CADENAS**

Se denomina **palabra, cadena, sarta o hilera** (formada con los símbolos de un alfabeto) a toda *secuencia finita de símbolos* de ese alfabeto. Ejemplos:

Cadenas sobre A1 : *JUAN, PEDRO, LUIS, AAA*
Cadenas sobre A2 : *0, 1, 11001100, 1111*
Cadenas sobre A3 : *2, 304, 126.34*
Cadenas sobre A4 : *\\\\///, $@@@*

CONVENCION → Se usarán letras minúsculas para representar las palabras de un alfabeto.
Ejemplos: x = JUAN (sobre A1) ; w = //\/ (sobre A4) ; z = 123.56 (sobre A3)

La **longitud** de una cadena se determina por el número de símbolos que la componen.
Así: |x| = 4 o lg(x) = 4 ; |w| = 4 ; lg(z) = 6
Se define la **cadena vacía o nula** como aquella cuya longitud es cero. Se representa mediante la letra **λ (lambda)**. Cualquiera que sea el alfabeto considerado, siempre puede formarse con sus símbolos la cadena vacía (bastaría formar una palabra sin seleccionar a ninguno de ellos).

CONVENCION → λ no será parte de ningún alfabeto.

El conjunto de cadenas que se pueden formar con los símbolos de un alfabeto recibe el nombre de "*lenguaje universal*" sobre ese alfabeto.
Sobre el alfabeto A, el lenguaje universal se representa por **A***.

Señala la respuesta correcta

- ✓ Evidentemente A* es un conjunto infinito
- ✓ Evidentemente A* es un finito.

λ pertenece a todos los lenguajes universales de todos los alfabetos posibles.

☞ OPERACIONES CON CADENAS

❖ Concatenación

Sean dos palabras x,w tales que: **x** $\in$ A*, **w** $\in$ A*

Supongamos que | x | = i y que | w | = j. Así **x** será $a_m a_{m2} a_{m3} \ldots a_{mi}$ y **w** será $a_{n1} a_{n2} a_{n3} \ldots a_{nj}$. Se llama **concatenación** de las palabras **x** y **w** (y se representa por xw o x.w) a otra palabra (z) obtenida poniendo los símbolos de w a continuación de las de x:

$z = a_{m1} a_{m2} a_{m3} \ldots a_{mi}\, a_{n1} a_{n2} a_{n3} \ldots a_{nj}$

Consideramos a continuación las **propiedades** de la concatenación:

- ✓ *Operación cerrada.* Es decir, la concatenación de dos palabras de A* es otra palabra de A*.
  Si x $\in$ A* e y $\in$ A*, entonces xy $\in$ A*
- ✓ *Propiedad asociativa.* $\forall$x, y, z se cumple que x(yz) = (xy)z
- ✓ *Existencia de elemento neutro.* El elemento neutro es la cadena nula λ, tanto por la derecha como por la izquierda. $\forall$x $x\lambda = \lambda x = x$
- ✓ *No cumple la propiedad conmutativa.* Esto se observa mediante cualquier contraejemplo. Sean las palabras x= ABC e y = CDE. Tenemos que xy = ABCCDE mientras que yx = CDEABC
- ✓ La *función longitud* de una palabra tiene, respecto a la concatenación, propiedades semejantes a las de la función logaritmo respecto de la multiplicación: | xy | = | x | + | y |

❖ Potencia

Se trata de una nueva notación para reducir algunos casos de la operación anterior. Se denomina potencia i-ésima de una palabra a la concatenación consigo misma **i** veces. Al poseer la concatenación la operación asociativa, no es necesario especificar el orden en que efectuar las concatenaciones.

Esto es → x^i = xxx...xx (i veces)

$x^1 = x$

$x^{i+1} = x^i x = x x^i$ (i > 0)

$x^i x^j = x^{i+j}$ (i, j > 0)

Se define: $x^0 = \lambda$, $\forall$x

$| x^i | = i | x |$

❖ Trasposición

Sea $x = a_{n1} a_{n2} a_{n3} \ldots a_{nj}$, se denomina traspuesta o inversa de x, representado por x^{-1} o x^t o $x^{\sim}$ o x^R a $a_{nj} a_{n-1j} \ldots a_{n3} a_{n2} a_{n1}$. Es decir, esta palabra está formada por las mismas letras, pero ordenadas de forma inversa. La función longitud es invariante con respecto a la reflexión de palabras: $| x^{-1} | = | x |$

☞ LENGUAJES

Se denomina **lenguaje** sobre el alfabeto A a cualquier subconjunto del lenguaje universal A* → $\mathbf{L} \subset$ A *

El conjunto vacío, ϕ, es un subconjunto de A*, no debe confundirse con aquel que contiene únicamente a la palabra vacía. $| \phi | = 0$ y $| \{ \lambda \} | = 1$.

El alfabeto en sí puede considerarse como un lenguaje: el formado por todas las posibles palabras de un símbolo.

CONVENCION → $\{ \lambda \} = \Lambda$

☞ OPERACIONES CON LENGUAJES

❖ Unión

Sean dos lenguajes diferentes definidos sobre el mismo alfabeto $L_1 \subset A^*$ y $L_2 \subset A^*$.

Se denomina **unión** de ambos lenguajes, $L_1 \cup L_2$, al lenguaje definido así: $\{ x \mid x \in L_1 \text{ ó } x \in L_2 \}$ (es el conjunto formado por las cadenas de ambos lenguajes).

Consideramos a continuación las **propiedades** de la unión:

- ✓ *Operación cerrada.* La unión de dos lenguajes definidos sobre el mismo alfabeto será otro lenguaje definido sobre ese alfabeto.
- ✓ *Propiedad asociativa.* $(L_1 \cup L_2) \cup L_3 = L_1 \cup (L_2 \cup L_3)$
- ✓ *Existencia de elemento neutro.* $\forall L$, se cumple que $L \cup \phi = \phi \cup L = L$
- ✓ *Propiedad conmutativa.* $\forall L_1, L_2$, se verifica que $L_1 \cup L_2 = L_2 \cup L_1$
- ✓ *Propiedad de idempotencia.* $\forall L$, se verifica que $L \cup L = L$

❖ Concatenación

Consideremos dos lenguajes definidos sobre el mismo alfabeto, L_1 y L_2. Se denomina **concatenación** de estos lenguajes al lenguaje definido así: $\{ xy \mid x \in L_1 \text{ y } y \in L_2 \}$ (todas las palabras de este lenguaje estarán formadas al concatenar cada una palabra del primero de los lenguajes con otra del segundo).

La definición anterior sólo es válida si L_1 y L_2 contienen al menos un elemento. La extensión de la concatenación de lenguajes al lenguaje vacío se hace de la siguiente manera: $\phi L = L\phi = \phi$

Con respecto a las **propiedades** de la concatenación:

- ✓ *Operación cerrada.* La concatenación de lenguajes sobre el mismo alfabeto es otro lenguaje sobre ese alfabeto.
- ✓ *Propiedad asociativa.* $(L_1 L_2) L_3 = L_1 (L_2 L_3)$
- ✓ *Elemento neutro (Λ).* $\forall L$, se cumple que $\Lambda L = L\Lambda = L$

❖ Potencia

Al igual que ocurre con esta operación sobre palabras, se trata de una notación para reducir algunos casos de la operación anterior. Se define la potencia i-ésima de un lenguaje a la operación de concatenarlo consigo mismo i veces. Al poseer esta operación la propiedad asociativa, no es necesario especificar el orden en que se efectuarán las operaciones. Esto es → $\mathbf{L^i = LLL \ldots L}$

$L^1 = L$

$L^{i+1} = L^i L = L L^i \quad (i > 0)$

$L^i L^j = L^{i+j} \quad (i, j > 0)$

Se define $L^0 = \{\lambda\} = \Lambda, \ \forall L$

❖ Clausura positiva

Se define la **clausura positiva** de un lenguaje L como:

$$\mathbf{L^+ = \bigcup_{i=1}^{\infty} L^i}$$

Se trata del lenguaje obtenido uniendo el lenguaje con todas sus potencias posibles, excepto L^0. Si L no contiene la palabra vacía, la clausura positiva tampoco la contendrá.

Ya que cualquier alfabeto A es un lenguaje sobre él mismo (formado por las palabras de longitud 1), se le puede aplicar esta operación. Se observa entonces que: $A^+ = A^* - \{\lambda\}$

- **Iteración, cierre o clausura**

  Se define la iteración, cierre o **clausura** de un lenguaje L como:

$$\mathbf{L^* = \bigcup_{i=0}^{\infty} L^i}$$

Se trata del lenguaje obtenido uniendo el lenguaje con todas sus potencias posibles, incluso L^0. Obviamente L* contiene siempre la cadena nula.
Son evidentes las siguientes relaciones:

$L^* = L^+ \cup \{\lambda\}$

$L^+ = L\,L^* = L^*\,L$

- **Trasposición**

  Se llama lenguaje **traspuesto** o **inverso** de L, representándose por L^{-1} o L^T a $\mathbf{\{\, x^{-1} \mid x \in L \,\}}$. Es decir, se trata de un lenguaje que contiene las palabras inversas a las palabras de L.

GRAMATICAS

Una gramática describe la estructura de las frases y palabras de un lenguaje a través del uso de reglas definidas

Una gramática $G = < V_N, V_T , S, P >$ consta de 4 elementos ordenados dónde:
$V_N = \{A,B,C...\}$ es un conjunto finito de **símbolos no terminales** (variables).
$V_T = \{a, b, c...\}$ es un conjunto finito de **símbolos terminales**.
$S \in V_N$ es el **símbolo inicial** de la gramática.
P es un conjunto finito de **reglas de derivación** o **reglas de reescritura** o **producciones** (indican cómo sustituir variables) de la forma $x \rightarrow y$ con $x, y \in V^*$

Se cumple que $V_N \cap V_T = \phi$

Se define $V = V_N \cup V_T$

Cadenas generadas por la gramática: se obtienen partiendo del símbolo inicial **S** mediante la aplicación sucesiva de reglas de **P**.

☞ **DERIVACIÓN**

Si $\alpha, \beta \in V^*$, se dice que $\alpha \rightarrow \beta$ ("β deriva de α") si existe alguna regla $\gamma ::= \delta \in P$ tal que $\alpha = \alpha_1 \gamma \alpha_2$ y $\beta = \alpha_1 \delta \alpha_2$, con $\alpha_1, \alpha_2 \in V^*$

Se indica con $\overset{*}{\rightarrow}$ la aplicación de una o más operaciones de **derivación.**

Se dice que una palabra α es una **forma sentencial** si $S \overset{*}{\rightarrow} \alpha$ con $\alpha \in V^*$

Lenguaje generado por la gramática $L(G) = \{ \omega \in V_T^* \ / \ S \overset{*}{\rightarrow} \omega \}$

Dos gramáticas G y G' se dicen **equivalentes** ($G \equiv G'$) si generan el mismo lenguaje, es decir, si $L(G) = L(G')$

Ejemplos:

G1 = < { 0, 1} , {S, B} , S , { S ::= 1B , B ::= 00 | λ} >

G2 = < { 0, 1} , {S, B} , S, { S ::= 1B | 1, B ::= 00} >

G1 y G2 son equivalentes

☞ ÁRBOLES DE DERIVACIÓN

A toda secuencia de derivación de una gramática le corresponde un **árbol de derivación,** que se construye así:

- ✓ La raíz del árbol corresponde al símbolo no terminal inicial
- ✓ Una derivación directa se representa por un conjunto de ramas que salen de un nodo determinado. Al aplicar una regla, uno de los símbolos de la parte izquierda de la producción queda sustituido por la parte derecha. Por cada uno de los símbolos de se dibuja una rama, que parte del nodo correspondiente al símbolo sustituido.

A lo largo del proceso de construcción del árbol, los nodos finales de cada paso, leídos de izquierda a derecha, forman la forma sentencial obtenida por la derivación representada por el árbol.

☞ AMBIGÜEDAD

En algunas gramáticas, una misma cadena/sentencia puede obtenerse como resultado de varias derivaciones diferentes, pero a las que les corresponde un único árbol de derivación.

Pero hay otros casos en que para una misma cadena/sentencia podemos tener varios árboles de derivación diferentes. En este caso, se dice que la gramática es **ambigua**.

Hay lenguajes para los cuales es imposible encontrar gramáticas no ambiguas. Estos lenguajes se denominan **inherentemente ambiguos**.